

M16 / PROTECTED ANSWERS

M16 Explained Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

DE16-01 | Production mindset - worked recovery 3

DE16-02 | Testing strategy for data engineering - worked recovery 4

DE16-03 | Automated quality gates - worked recovery 4

DE16-04 | Continuous Integration fundamentals - worked recovery 5

DE16-05 | GitHub Actions for data projects - worked recovery 6

DE16-06 | Dev, test and prod environments - worked recovery 7

DE16-07 | Deployment and rollback - worked recovery 8

DE16-08 | Secrets and credential safety - worked recovery 9

DE16-09 | RBAC and least privilege - worked recovery 10

DE16-10 | PII, encryption and masking - worked recovery 11

DE16-11 | Metadata, lineage and governance - worked recovery 11

DE16-12 | Observability for data pipelines - worked recovery 12

DE16-13 | Incidents, runbooks and root cause - worked recovery 13

DE16-14 | Terraform and IaC foundations - worked recovery 14

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open only after a genuine attempt. These are worked recovery explanations, not answer-before-attempt notes.

DE16-01 | Production mindset - worked recovery

What a strong attempt proves. You should finish with a small production contract where another engineer can tell what healthy means, where to look for evidence, and which failure should stop publication.

Typical weak attempt. At 05:40 the orchestration UI says success, but the gold table contains only 12% of the normal rows because the source API changed pagination. Compute succeeded; the business contract failed. A row-count/reconciliation gate must stop publication and preserve the previous good state.

Recovery reasoning. Start by preserving the failing evidence and naming the violated boundary. Separate code/configuration state from business/data state and from identity/permission state. Repair the narrowest cause that explains the evidence; do not broaden privilege or delete rejected data simply to obtain a green run. Then rerun the original validation and a changed case so you know the repair generalizes.

Worked decision trace. Return to the learner demonstration: Take a nightly orders pipeline. Write five promises before choosing tools: every accepted order appears once in curated data; paid revenue reconciles to source; data is ready by 06:00; secrets never appear in code/logs; and a bad release can be rolled back without deleting accepted source evidence. Map each promise to one independent proof. A green scheduler icon is not one of those proofs. Now compare that controlled route with the failure. The key is to locate the first boundary where observed evidence stops matching the stated contract. Keep earlier valid evidence intact, make one reversible change, and re-run the exact check that first proved the failure. Only after that passes should you perform the fresh retry.

Why common shortcuts are unsafe. A shortcut such as granting broad access, ignoring a failed check, deleting a quarantined batch, manually editing production state, or rerunning until the dashboard looks normal can hide the symptom without proving the system is correct. A professional recovery leaves an auditable chain: original failure -> bounded diagnosis -> controlled repair -> original validation -> changed validation.

Validation checklist. You should finish with a small production contract where another engineer can tell what healthy means, where to look for evidence, and which failure should stop publication. In addition, confirm that the evidence comes from the intended environment, that the first failed attempt was not overwritten, and that the repaired state can be explained without referring to an answer key.

Changed retry you should be able to do cold. Change the case to a streaming payments feed. Decide which promises need tighter latency, what replay/deduplication evidence is required, and what can remain eventually consistent.

Teach-back check. Explain why 'the DAG is green' is not enough to call a data product healthy. Give two examples of successful execution with wrong business output.

DE16-02 | Testing strategy for data engineering - worked recovery

What a strong attempt proves. A test matrix that names the test layer, fixture, command, expected result, runtime/cost, and defect class. Fast tests belong on every pull request; expensive environment checks belong at stronger release boundaries.

Typical weak attempt. A refactor passes 65 unit tests but doubles daily revenue because a join changed grain and created fan-out. The missing proof was not another helper-function test; it was a grain/reconciliation assertion at the data boundary.

Recovery reasoning. Start by preserving the failing evidence and naming the violated boundary. Separate code/configuration state from business/data state and from identity/permission state. Repair the narrowest cause that explains the evidence; do not broaden privilege or delete rejected data simply to obtain a green run. Then rerun the original validation and a changed case so you know the repair generalizes.

Worked decision trace. Return to the learner demonstration: For order totals, first test the pure aggregation function with three rows. Then check required columns/types and non-null business keys. Next write the result to a disposable database. Finally reconcile source quantity*price against the curated sum. Each layer should tell you what class of defect it can catch and what it cannot prove. Now compare that controlled route with the failure. The key is to locate the first boundary where observed evidence stops matching the stated contract. Keep earlier valid evidence intact, make one reversible change, and re-run the exact check that first proved the failure. Only after that passes should you perform the fresh retry.

Why common shortcuts are unsafe. A shortcut such as granting broad access, ignoring a failed check, deleting a quarantined batch, manually editing production state, or rerunning until the dashboard looks normal can hide the symptom without proving the system is correct. A professional recovery leaves an auditable chain: original failure -> bounded diagnosis -> controlled repair -> original validation -> changed validation.

Validation checklist. A test matrix that names the test layer, fixture, command, expected result, runtime/cost, and defect class. Fast tests belong on every pull request; expensive environment checks belong at stronger release boundaries. In addition, confirm that the evidence comes from the intended environment, that the first failed attempt was not overwritten, and that the repaired state can be explained without referring to an answer key.

Changed retry you should be able to do cold. The source adds nullable coupon_code. Decide what should continue to pass, what would fail if the field became required, and how backwards compatibility is demonstrated.

Teach-back check. Using one orders example, distinguish a unit test, a data-quality expectation, an integration test and a reconciliation check.

DE16-03 | Automated quality gates - worked recovery

What a strong attempt proves. A result containing rule name, measured value, threshold, severity and final decision. On a hard failure the committed accepted checkpoint must remain unchanged.

Typical weak attempt. A source sends half the normal rows after an upstream timeout. Schema and uniqueness pass, but volume fails. If you advance the checkpoint before validation, replay becomes harder and bad progress is now accepted state. Correct behavior is quarantine + alert + no publish + no checkpoint advance.

Recovery reasoning. Start by preserving the failing evidence and naming the violated boundary. Separate code/configuration state from business/data state and from identity/permission state. Repair the narrowest cause that explains the evidence; do not broaden privilege or delete rejected data simply to obtain a green run. Then rerun the original validation and a changed case so you know the repair generalizes.

Worked decision trace. Return to the learner demonstration: For a customer batch define four rules: non-empty; customer_id unique/non-null; country in allowed set; row count at least 80% of rolling baseline. Run a good fixture and then a duplicate-ID fixture. The gate must emit machine-readable evidence, refuse publication on the hard failure, and retain the rejected batch for diagnosis. Now compare that controlled route with the failure. The key is to locate the first boundary where observed evidence stops matching the stated contract. Keep earlier valid evidence intact, make one reversible change, and re-run the exact check that first proved the failure. Only after that passes should you perform the fresh retry.

Why common shortcuts are unsafe. A shortcut such as granting broad access, ignoring a failed check, deleting a quarantined batch, manually editing production state, or rerunning until the dashboard looks normal can hide the symptom without proving the system is correct. A professional recovery leaves an auditable chain: original failure -> bounded diagnosis -> controlled repair -> original validation -> changed validation.

Validation checklist. A result containing rule name, measured value, threshold, severity and final decision. On a hard failure the committed accepted checkpoint must remain unchanged. In addition, confirm that the evidence comes from the intended environment, that the first failed attempt was not overwritten, and that the repaired state can be explained without referring to an answer key.

Changed retry you should be able to do cold. Change the volume baseline by 20% and show why the same incoming batch changes decision only because the contractual threshold changed. Record the rule version.

Teach-back check. Why is a validation report alone not a quality gate? Name the additional control that makes it a gate.

DE16-04 | Continuous Integration fundamentals - worked recovery

What a strong attempt proves. Every required check either passes or stops the workflow with a non-zero decision. A reviewer can see which gate failed without rerunning your laptop setup.

Typical weak attempt. Common mistakes are running CI only after merge, hiding errors with continue-on-error, using floating dependencies, or putting powerful production credentials into a test workflow that never needed them. Workflow files themselves are executable code and must be reviewed as a trust boundary.

Recovery reasoning. Start by preserving the failing evidence and naming the violated boundary. Separate code/configuration state from business/data state and from identity/permission state. Repair the narrowest cause that explains the evidence; do not broaden privilege or delete

rejected data simply to obtain a green run. Then rerun the original validation and a changed case so you know the repair generalizes.

Worked decision trace. Return to the learner demonstration: Declare a minimal flow: checkout -> set up Python -> install locked dependencies -> compile -> unit tests -> fixture quality checks -> secret-pattern scan. Deliberately fail unit tests. The simulator should prove that later release steps are unreachable, then identify the failed stage in the evidence. Now compare that controlled route with the failure. The key is to locate the first boundary where observed evidence stops matching the stated contract. Keep earlier valid evidence intact, make one reversible change, and re-run the exact check that first proved the failure. Only after that passes should you perform the fresh retry.

Why common shortcuts are unsafe. A shortcut such as granting broad access, ignoring a failed check, deleting a quarantined batch, manually editing production state, or rerunning until the dashboard looks normal can hide the symptom without proving the system is correct. A professional recovery leaves an auditable chain: original failure -> bounded diagnosis -> controlled repair -> original validation -> changed validation.

Validation checklist. Every required check either passes or stops the workflow with a non-zero decision. A reviewer can see which gate failed without rerunning your laptop setup. In addition, confirm that the evidence comes from the intended environment, that the first failed attempt was not overwritten, and that the repaired state can be explained without referring to an answer key.

Changed retry you should be able to do cold. First make the unit-test stage fail and prove deploy is unreachable. Then fix it and introduce a secret-pattern failure so a different gate blocks the path.

Teach-back check. Explain CI to someone who believes it means 'automatically deploy every commit.' What is missing from that definition?

DE16-05 | GitHub Actions for data projects - worked recovery

What a strong attempt proves. You can identify trigger, job, runner image, reusable actions, shell commands, permissions and fail-closed checks. You can also explain which steps could run safely on an untrusted pull request.

Typical weak attempt. A workflow uses a long-lived cloud key stored as a secret even though the provider supports OIDC. The key is copied into a debug command and appears in logs. The correct design is to remove unnecessary privilege, rotate/revoke leaked credentials, and prefer short-lived federated identity with explicit trust conditions.

Recovery reasoning. Start by preserving the failing evidence and naming the violated boundary. Separate code/configuration state from business/data state and from identity/permission state. Repair the narrowest cause that explains the evidence; do not broaden privilege or delete rejected data simply to obtain a green run. Then rerun the original validation and a changed case so you know the repair generalizes.

Worked decision trace. Return to the learner demonstration: Read the supplied workflow. It runs on pull_request and manual_workflow_dispatch, grants contents: read, checks out code, sets up Python, compiles the practice folder and runs the quality gate. Notice that test CI needs no production secret. Then inspect where permissions are declared and why adding write-all would

increase blast radius. Now compare that controlled route with the failure. The key is to locate the first boundary where observed evidence stops matching the stated contract. Keep earlier valid evidence intact, make one reversible change, and re-run the exact check that first proved the failure. Only after that passes should you perform the fresh retry.

Why common shortcuts are unsafe. A shortcut such as granting broad access, ignoring a failed check, deleting a quarantined batch, manually editing production state, or rerunning until the dashboard looks normal can hide the symptom without proving the system is correct. A professional recovery leaves an auditable chain: original failure -> bounded diagnosis -> controlled repair -> original validation -> changed validation.

Validation checklist. You can identify trigger, job, runner image, reusable actions, shell commands, permissions and fail-closed checks. You can also explain which steps could run safely on an untrusted pull request. In addition, confirm that the evidence comes from the intended environment, that the first failed attempt was not overwritten, and that the repaired state can be explained without referring to an answer key.

Changed retry you should be able to do cold. Add a second job that depends on validate and produces a harmless text artifact. Break validate and prove the artifact job does not run; then repair it.

Teach-back check. What is the difference between `uses:` and `run:`? Why should workflow permissions be designed explicitly?

DE16-06 | Dev, test and prod environments - worked recovery

What a strong attempt proves. You can show the same artifact/version across environments while endpoints, identities and capacities change through explicit configuration. No secret value is baked into the artifact.

Typical weak attempt. A test pipeline is copied manually from production and still references the prod sink. A harmless test run overwrites business data. The root cause is not 'someone clicked wrong'; it is missing environment binding controls and excessive test identity permissions.

Recovery reasoning. Start by preserving the failing evidence and naming the violated boundary. Separate code/configuration state from business/data state and from identity/permission state. Repair the narrowest cause that explains the evidence; do not broaden privilege or delete rejected data simply to obtain a green run. Then rerun the original validation and a changed case so you know the repair generalizes.

Worked decision trace. Return to the learner demonstration: Use one pipeline artifact and three configuration maps. Dev reads synthetic/local data, test reads a controlled integration source, prod reads the approved production source. The binding checker rejects a configuration where the test artifact points at the production customer table. This is environment safety expressed as code. Now compare that controlled route with the failure. The key is to locate the first boundary where observed evidence stops matching the stated contract. Keep earlier valid evidence intact, make one reversible change, and re-run the exact check that first proved the failure. Only after that passes should you perform the fresh retry.

Why common shortcuts are unsafe. A shortcut such as granting broad access, ignoring a failed check, deleting a quarantined batch, manually editing production state, or rerunning until the dashboard looks normal can hide the

symptom without proving the system is correct. A professional recovery leaves an auditable chain: original failure -> bounded diagnosis -> controlled repair -> original validation -> changed validation.

Validation checklist. You can show the same artifact/version across environments while endpoints, identities and capacities change through explicit configuration. No secret value is baked into the artifact. In addition, confirm that the evidence comes from the intended environment, that the first failed attempt was not overwritten, and that the repaired state can be explained without referring to an answer key.

Changed retry you should be able to do cold. Swap one test binding to a production resource and prove your checker rejects it. Then add an approved read-only prod reference and explain why its risk differs.

Teach-back check. Why should promotion prefer the same artifact with different bindings rather than three manually edited copies?

DE16-07 | Deployment and rollback - worked recovery

What a strong attempt proves. A deployment record naming version, environment, checks, decision, rollback target and affected data window. Rollback restores code safely without pretending data repair is automatic.

Typical weak attempt. A destructive schema migration drops a column before all consumers are updated. Rolling application code back cannot recreate the deleted values. Safer changes often use expand-contract: add compatible structure, migrate/verify readers, then remove old structure in a later controlled release.

Recovery reasoning. Start by preserving the failing evidence and naming the violated boundary. Separate code/configuration state from business/data state and from identity/permission state. Repair the narrowest cause that explains the evidence; do not broaden privilege or delete rejected data simply to obtain a green run. Then rerun the original validation and a changed case so you know the repair generalizes.

Worked decision trace. Return to the learner demonstration: Simulate release v2. Smoke tests pass, but the first production reconciliation fails. Mark v2 unhealthy, restore v1 code, identify rows written by v2, and decide whether they can be deterministically replayed. The evidence should distinguish code state from data state. Now compare that controlled route with the failure. The key is to locate the first boundary where observed evidence stops matching the stated contract. Keep earlier valid evidence intact, make one reversible change, and re-run the exact check that first proved the failure. Only after that passes should you perform the fresh retry.

Why common shortcuts are unsafe. A shortcut such as granting broad access, ignoring a failed check, deleting a quarantined batch, manually editing production state, or rerunning until the dashboard looks normal can hide the symptom without proving the system is correct. A professional recovery leaves an auditable chain: original failure -> bounded diagnosis -> controlled repair -> original validation -> changed validation.

Validation checklist. A deployment record naming version, environment, checks, decision, rollback target and affected data window. Rollback restores code safely without pretending data repair is automatic. In addition, confirm that the evidence comes from the intended environment,

that the first failed attempt was not overwritten, and that the repaired state can be explained without referring to an answer key.

Changed retry you should be able to do cold. Change the case so v2 writes an idempotent derived table rather than destructive state. Explain why replay is now easier and what evidence still matters.

Teach-back check. Why is code rollback not the same as data rollback? Give one example where the old code is restored but the system remains wrong.

DE16-08 | Secrets and credential safety - worked recovery

What a strong attempt proves. You can explain where secret material may leak (code, git history, logs, notebooks, config exports) and what to do after exposure: revoke/rotate, investigate use, remove from future code/history where appropriate, and replace the design.

Typical weak attempt. A key is accidentally committed and then 'fixed' by deleting the line in the next commit. The key still exists in history and may already have been copied. Treat it as compromised immediately; revoke/rotate first. Cleanup is secondary to removing authority.

Recovery reasoning. Start by preserving the failing evidence and naming the violated boundary. Separate code/configuration state from business/data state and from identity/permission state. Repair the narrowest cause that

explains the evidence; do not broaden privilege or delete rejected data simply to obtain a green run. Then rerun the original validation and a changed case so you know the repair generalizes.

Worked decision trace. Return to the learner demonstration: Run the hygiene scanner over sample configuration. It should flag obvious embedded tokens but not treat the scanner as complete protection. Then sketch a GitHub Actions -> cloud OIDC flow: workflow identity token, cloud trust policy, short-lived role/session, least-privilege access. No long-lived cloud key is committed to the repository. Now compare that controlled route with the failure. The key is to locate the first boundary where observed evidence stops matching the stated contract. Keep earlier valid evidence intact, make one reversible change, and re-run the exact check that first proved the failure. Only after that passes should you perform the fresh retry.

Why common shortcuts are unsafe. A shortcut such as granting broad access, ignoring a failed check, deleting a quarantined batch, manually editing production state, or rerunning until the dashboard looks normal can hide the symptom without proving the system is correct. A professional recovery leaves an auditable chain: original failure -> bounded diagnosis -> controlled repair -> original validation -> changed validation.

Validation checklist. You can explain where secret material may leak (code, git history, logs, notebooks, config exports) and what to do after exposure: revoke/rotate, investigate use, remove from future code/history where appropriate, and replace the design. In addition, confirm that the evidence comes from the intended environment, that the first failed attempt was not overwritten, and that the repaired state can be explained without referring to an answer key.

Changed retry you should be able to do cold. Assume OIDC is temporarily unavailable for one legacy service. Design a contained secret fallback with rotation, restricted scope, masked logs and an explicit removal date.

Teach-back check. Why does deleting a leaked credential from the latest source file not make the credential safe again?

DE16-09 | RBAC and least privilege - worked recovery

What a strong attempt proves. A role matrix where principal, action, scope and justification are explicit. Broad roles have documented exception reasoning rather than being default troubleshooting tools.

Typical weak attempt. A pipeline receives 403 and the quickest fix is to grant Owner at subscription scope. The job works, but the identity can now change far more than necessary. Correct debugging identifies the exact denied operation and grants the narrow data/control permission at the smallest workable scope.

Recovery reasoning. Start by preserving the failing evidence and naming the violated boundary. Separate code/configuration state from business/data state and from identity/permission state. Repair the narrowest cause that explains the evidence; do not broaden privilege or delete rejected data simply to obtain a green run. Then rerun the original validation and a changed case so you know the repair generalizes.

Worked decision trace. Return to the learner demonstration: Model three principals: ingestion identity writes raw storage, transformation identity reads raw/writes curated, analyst identity reads curated. The policy checker rejects Owner at subscription scope for all three. Then compare a management role with a data-access role: being able to configure a Key Vault is not automatically the same as being allowed to read every secret. Now compare that controlled route with the failure. The key is to locate the first boundary where observed evidence stops matching the stated contract. Keep earlier valid evidence intact, make one reversible change, and re-run the exact check that first proved the failure. Only after that passes should you perform the fresh retry.

Why common shortcuts are unsafe. A shortcut such as granting broad access, ignoring a failed check, deleting a quarantined batch, manually editing production state, or rerunning until the dashboard looks normal can hide the symptom without proving the system is correct. A professional recovery leaves an auditable chain: original failure -> bounded diagnosis -> controlled repair -> original validation -> changed validation.

Validation checklist. A role matrix where principal, action, scope and justification are explicit. Broad roles have documented exception reasoning rather than being default troubleshooting tools. In addition, confirm that the evidence comes from the intended environment, that the first failed attempt was not overwritten, and that the repaired state can be explained without referring to an answer key.

Changed retry you should be able to do cold. Change the transformation task so it needs to rotate a secret. Decide whether to expand its existing role or delegate rotation to a separate automation identity.

Teach-back check. Explain principal + role + scope with one Azure example. Why can a management-plane role differ from data access?

DE16-10 | PII, encryption and masking - worked recovery

What a strong attempt proves. You can classify fields, identify raw versus masked zones, state who needs unmasked access, and explain how logs/telemetry avoid recording PII values.

Typical weak attempt. A pipeline correctly encrypts storage but prints full email addresses and authentication headers to logs during debugging. The logs are copied to a wider observability platform. Encryption at rest did not prevent the exposure because the sensitive values were intentionally emitted after decryption.

Recovery reasoning. Start by preserving the failing evidence and naming the violated boundary. Separate code/configuration state from business/data state and from identity/permission state. Repair the narrowest cause that explains the evidence; do not broaden privilege or delete rejected data simply to obtain a green run. Then rerun the original validation and a changed case so you know the repair generalizes.

Worked decision trace. Return to the learner demonstration: Load a safe customer fixture containing synthetic email and phone values. The masking function emits a support-safe view while preserving a stable non-PII customer key. Compare four controls: source minimization, storage encryption, access control and output masking. None substitutes for the others. Now compare that controlled route with the failure. The key is to locate the first boundary where observed evidence stops matching the stated contract. Keep earlier valid evidence intact, make one reversible change, and re-run the exact check that first proved the failure. Only after that passes should you perform the fresh retry.

Why common shortcuts are unsafe. A shortcut such as granting broad access, ignoring a failed check, deleting a quarantined batch, manually editing production state, or rerunning until the dashboard looks normal can hide the

symptom without proving the system is correct. A professional recovery leaves an auditable chain: original failure -> bounded diagnosis -> controlled repair -> original validation -> changed validation.

Validation checklist. You can classify fields, identify raw versus masked zones, state who needs unmasked access, and explain how logs/telemetry avoid recording PII values. In addition, confirm that the evidence comes from the intended environment, that the first failed attempt was not overwritten, and that the repaired state can be explained without referring to an answer key.

Changed retry you should be able to do cold. Add a legitimate fraud-use case requiring a small group to see raw phone numbers. Define the narrow access, audit and retention controls rather than weakening every dataset.

Teach-back check. Why are encryption and masking different controls? Give one failure that encryption alone does not prevent.

DE16-11 | Metadata, lineage and governance - worked recovery

What a strong attempt proves. A lineage graph that supports both root-cause questions ('where did this come from?') and impact questions ('what will break if I change this?'). Known lineage gaps are stated instead of hidden.

Typical weak attempt. A catalog lists every table but no owners, transformation relationships or sensitive classifications. Search works, but incident routing and impact analysis do not.

Governance requires usable metadata and operating responsibility, not just inventory count.

Recovery reasoning. Start by preserving the failing evidence and naming the violated boundary. Separate code/configuration state from business/data state and from identity/permission state. Repair the narrowest cause that explains the evidence; do not broaden privilege or delete rejected data simply to obtain a green run. Then rerun the original validation and a changed case so you know the repair generalizes.

Worked decision trace. Return to the learner demonstration: Build a graph: orders_api -> raw_orders -> silver_orders -> gold_daily_sales -> executive_dashboard. Attach owner, grain and sensitivity to each node. Then simulate removing discount_amount from silver. Traverse downstream edges and list assets needing review before the change is approved. Now compare that controlled route with the failure. The key is to locate the first boundary where observed evidence stops matching the stated contract. Keep earlier valid evidence intact, make one reversible change, and re-run the exact check that first proved the failure. Only after that passes should you perform the fresh retry.

Why common shortcuts are unsafe. A shortcut such as granting broad access, ignoring a failed check, deleting a quarantined batch, manually editing production state, or rerunning until the dashboard looks normal can hide the symptom without proving the system is correct. A professional recovery leaves an auditable chain: original failure -> bounded diagnosis -> controlled repair -> original validation -> changed validation.

Validation checklist. A lineage graph that supports both root-cause questions ('where did this come from?') and impact questions ('what will break if I change this?'). Known lineage gaps are stated instead of hidden. In addition, confirm that the evidence comes from the intended environment, that the first failed attempt was not overwritten, and that the repaired state can be explained without referring to an answer key.

Changed retry you should be able to do cold. Insert a second consumer and a shortcut/external table whose lineage is only partially captured. Mark the gap and document a manual control until automated lineage is available.

Teach-back check. What can lineage answer that a schema alone cannot? Give one upstream root-cause and one downstream impact question.

DE16-12 | Observability for data pipelines - worked recovery

What a strong attempt proves. A compact health record with software and data signals, enough context to route an incident, and no raw PII as high-cardinality telemetry labels.

Typical weak attempt. Teams emit customer_email as a metric label to debug individual failures. This leaks PII and creates huge cardinality/cost. Telemetry should use bounded technical identifiers and link to appropriately secured investigation data when necessary.

Recovery reasoning. Start by preserving the failing evidence and naming the violated boundary. Separate code/configuration state from business/data state and from identity/permission state. Repair the narrowest cause that

explains the evidence; do not broaden privilege or delete rejected data simply to obtain a green run. Then rerun the original validation and a changed case so you know the repair generalizes.

Worked decision trace. Return to the learner demonstration: Generate a health snapshot containing run duration, error count, output rows, freshness minutes and reconciliation delta. Keep runtime normal but drop output rows by 80%. Software telemetry alone looks fine; data-volume/reconciliation telemetry reveals the real failure. Add an alert decision that names both measured value and expected boundary. Now compare that controlled route with the failure. The key is to locate the first boundary where observed evidence stops matching the stated contract. Keep earlier valid evidence intact, make one reversible change, and re-run the exact check that first proved the failure. Only after that passes should you perform the fresh retry.

Why common shortcuts are unsafe. A shortcut such as granting broad access, ignoring a failed check, deleting a quarantined batch, manually editing production state, or rerunning until the dashboard looks normal can hide the symptom without proving the system is correct. A professional recovery leaves an auditable chain: original failure -> bounded diagnosis -> controlled repair -> original validation -> changed validation.

Validation checklist. A compact health record with software and data signals, enough context to route an incident, and no raw PII as high-cardinality telemetry labels. In addition, confirm that the evidence comes from the intended environment, that the first failed attempt was not overwritten, and that the repaired state can be explained without referring to an answer key.

Changed retry you should be able to do cold. Change the incident so row count is normal but freshness is six hours late. Decide which alert should fire and why a pure volume dashboard would miss it.

Teach-back check. Why can CPU/runtime/success metrics all be healthy while a data product is unhealthy? Name three data-specific signals.

DE16-13 | Incidents, runbooks and root cause - worked recovery

What a strong attempt proves. A timeline separating observed facts from hypotheses, containment action, recovery proof, customer/data impact, causal chain and prevention tasks with owners.

Typical weak attempt. During an incident someone reruns the pipeline repeatedly with debug flags and deletes the failed batch to 'start clean.' This destroys evidence and may duplicate side effects. Safe response freezes relevant state, records commands and changes one variable at a time.

Recovery reasoning. Start by preserving the failing evidence and naming the violated boundary. Separate code/configuration state from business/data state and from identity/permission state. Repair the narrowest cause that explains the evidence; do not broaden privilege or delete rejected data simply to obtain a green run. Then rerun the original validation and a changed case so you know the repair generalizes.

Worked decision trace. Return to the learner demonstration: Use a scenario: gold revenue becomes stale after deployment. Timeline facts show source arrived, raw load completed, silver validation failed, publish gate held the previous gold version. The runbook guides evidence collection and safe replay. The RCA then identifies a schema assumption introduced in the release and adds a contract test to prevent recurrence. Now compare that controlled route with the failure. The key is to locate the first boundary where observed evidence stops matching the

stated contract. Keep earlier valid evidence intact, make one reversible change, and re-run the exact check that first proved the failure. Only after that passes should you perform the fresh retry.

Why common shortcuts are unsafe. A shortcut such as granting broad access, ignoring a failed check, deleting a quarantined batch, manually editing production state, or rerunning until the dashboard looks normal can hide the symptom without proving the system is correct. A professional recovery leaves an auditable chain: original failure -> bounded diagnosis -> controlled repair -> original validation -> changed validation.

Validation checklist. A timeline separating observed facts from hypotheses, containment action, recovery proof, customer/data impact, causal chain and prevention tasks with owners. In addition, confirm that the evidence comes from the intended environment, that the first failed attempt was not overwritten, and that the repaired state can be explained without referring to an answer key.

Changed retry you should be able to do cold. Change the symptom to high latency with correct output. Decide which containment and evidence steps differ and which remain the same.

Teach-back check. What is the difference between a runbook and an RCA? Why should facts and hypotheses be labeled separately during an incident?

DE16-14 | Terraform and IaC foundations - worked recovery

What a strong attempt proves. You can explain what config, provider, state, plan and apply each do; you do not call a successful plan proof that the data pipeline itself is correct.

Typical weak attempt. A team edits production resources manually after Terraform deploy. The next plan shows unexpected changes. Applying blindly can overwrite deliberate emergency fixes or propagate stale assumptions. First understand drift and ownership. State files can also contain sensitive values and require protected storage/access.

Recovery reasoning. Start by preserving the failing evidence and naming the violated boundary. Separate code/configuration state from business/data state and from identity/permission state. Repair the narrowest cause that explains the evidence; do not broaden privilege or delete rejected data simply to obtain a green run. Then rerun the original validation and a changed case so you know the repair generalizes.

Worked decision trace. Return to the learner demonstration: Read the local sample configuration. It declares a Terraform version range $\geq 1.16, < 1.17$ for this controlled exercise and uses no real cloud credential. Walk the safe reasoning route: `terraform fmt`, `terraform init`, `terraform validate`, `terraform plan`, human/policy review, then apply only in an approved environment. The policy checker flags patterns such as wildcard admin intent in teaching examples. Now compare that controlled route with the failure. The key is to locate the first boundary where observed evidence stops matching the stated contract. Keep earlier valid evidence intact, make one reversible change, and re-run the exact check that first proved the failure. Only after that passes should you perform the fresh retry.

Why common shortcuts are unsafe. A shortcut such as granting broad access, ignoring a failed check, deleting a quarantined batch, manually editing production state, or rerunning until the dashboard looks normal can hide the

symptom without proving the system is correct. A professional recovery leaves an auditable chain: original failure -> bounded diagnosis -> controlled repair -> original validation -> changed validation.

Validation checklist. You can explain what config, provider, state, plan and apply each do; you do not call a successful plan proof that the data pipeline itself is correct. In addition, confirm that the evidence comes from the intended environment, that the first failed attempt was not overwritten, and that the repaired state can be explained without referring to an answer key.

Changed retry you should be able to do cold. Change one harmless local variable in the sample configuration and predict the plan effect. Then imagine the same change affects a production storage account; list additional approvals and data-safety checks.

Teach-back check. Explain why terraform plan is valuable but does not prove application or data correctness. What extra evidence belongs after infrastructure deployment?